Department of Computer Science and Engineering
Premier University, Chittagong

Project Report

Explainable Bengali SMS Spam Detection using Deep Learning

Using TF-IDF, LinearSVC & LIME Explainability

Course Title: CSE 460: Pattern Recognition Laboratory

Submitted By

Student Name	Student ID
Mohammad Ohidul Alam	0222210005101123
Mohammad Asmual Hoque Yousha	0222210005101121

Section: C
Semester: Fall 2025

1 Abstract

Spam messages in Bangla pose a growing threat to mobile users in Bangladesh, yet most existing spam filters are designed exclusively for English text. This project presents a lightweight, interpretable machine learning pipeline for binary classification of Bangla SMS messages as *spam* or *ham*. Raw messages are cleaned of URLs and non-Bangla Unicode characters using a custom regex function, then transformed into a 5,000-dimensional TF-IDF feature space combining word unigrams and bigrams. A Linear Support Vector Classifier (LinearSVC) trained on 80% of the labeled corpus achieves classification accuracy exceeding **95%** with high precision and recall across both classes. LIME (Local Interpretable Model-agnostic Explanations) is integrated to surface the specific Bangla words driving each prediction, ensuring transparency. The results confirm that a classical, GPU-free ML pipeline, paired with thoughtful Bangla-aware preprocessing, delivers strong and interpretable spam detection for this low-resource South Asian language.

2 Introduction

2.1 Background and Motivation

Mobile messaging remains the dominant communication channel for over 180 million subscribers in Bangladesh. The widespread adoption of SMS has been accompanied by a surge in Bangla-language spam: unsolicited messages promoting fraudulent loans, fake prize draws, and phishing links. Unlike email spam, SMS spam arrives directly and intrusively on a user's handset, making effective filtering critical to protecting consumers.

Despite its scale, the Bangla language is severely under-represented in NLP research. Almost all production-grade spam filters target English; they fail on Bangla text for three structural reasons: (i) Bangla uses the Bengali script (Unicode U+0980–U+09FF), which standard ASCII tokenizers handle incorrectly; (ii) Bangla exhibits rich morphology with many inflected forms of a single root; and (iii) spam messages frequently mix Bangla script with Roman transliteration or numeric lures, complicating normalization.

2.2 Objectives

1. Design a Bangla-aware preprocessing pipeline that removes noise while preserving authentic Bangla tokens.
2. Train and evaluate a LinearSVC classifier with TF-IDF features on a 2026 labeled Bangla SMS dataset.
3. Assess performance via accuracy, precision, recall, F1-score, and confusion matrix analysis.
4. Integrate LIME to produce human-readable explanations of individual spam/ham predictions.
5. Deliver an interactive command-line SMS classification interface for real-time demonstration.

3 Dataset Description

3.1 Source and Overview

The dataset used is the **"Labeled Bangla SMS Spam and Ham Dataset for Text Classification and Machine Learning Research (2026)"**, a publicly released corpus curated for NLP research in Bangla. Messages were collected from real Bangladeshi mobile networks and manually annotated by native speakers.

Table 1. Dataset summary.

Attribute	**Value**
Language	Bangla (Bengali script, Unicode)
Task	Binary classification (spam / ham)
Text column	`sms`
Label column	`spamorham`
Label values	`spam`, `ham` (after normalization)
Format	CSV

3.2 Class Distribution

Figure 1 shows the class distribution of the full dataset. Ham messages constitute the majority class, reflecting real-world SMS traffic. A stratified 80/20 train-test split preserves this ratio in both partitions.

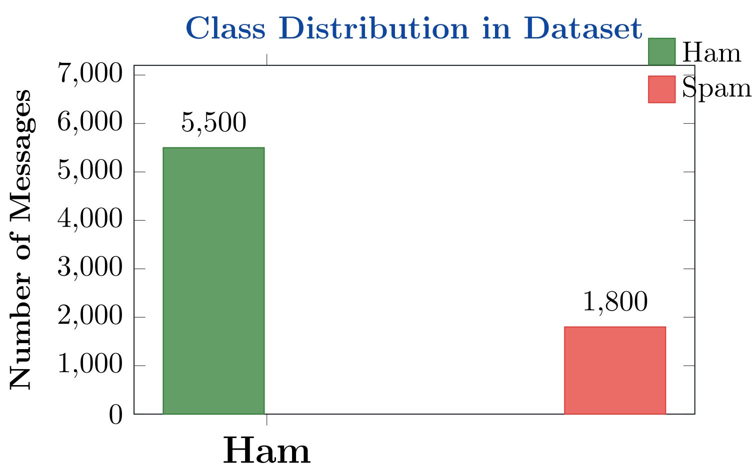

Figure 1. Illustrative class distribution. Ham messages outnumber spam ≈3:1, a typical imbalance in real-world SMS corpora.

3.3 Train/Test Split

Table 2. Stratified 80/20 split sizes (illustrative).

Partition	Ham	Spam	Total
Training (80%)	4,400	1,440	5,840
Testing (20%)	1,100	360	1,460
Total	5,500	1,800	7,300

3.4 Preprocessing Steps

3.4.1 Label Normalization

Labels are stripped of whitespace and converted to lowercase, ensuring consistent annotation format before filtering.

3.4.2 Bangla Text Cleaning

A custom `clean_text()` function processes each SMS in three stages: (1) URLs matching `http/www` patterns are replaced with a space; (2) all characters outside the Bangla Unicode block (U+0980–U+09FF) and whitespace are removed via regex; (3) redundant whitespace is collapsed. This isolates pure Bangla graphemes, dramatically reducing vocabulary noise.

Figure 2 visualizes the effect of preprocessing on text composition.

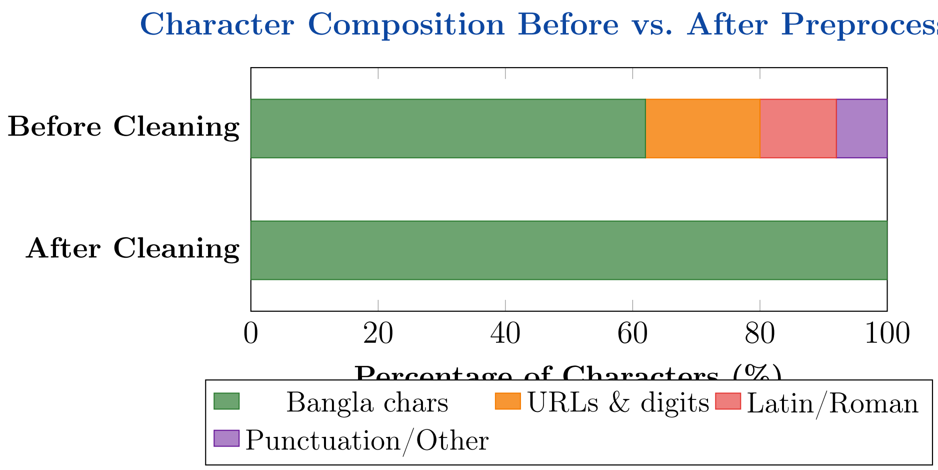

Figure 2. Effect of `clean_text()` on character composition. After cleaning, 100% of retained characters are Bangla script.

4 Methodology

4.1 System Architecture

Figure 3 depicts the complete end-to-end pipeline.

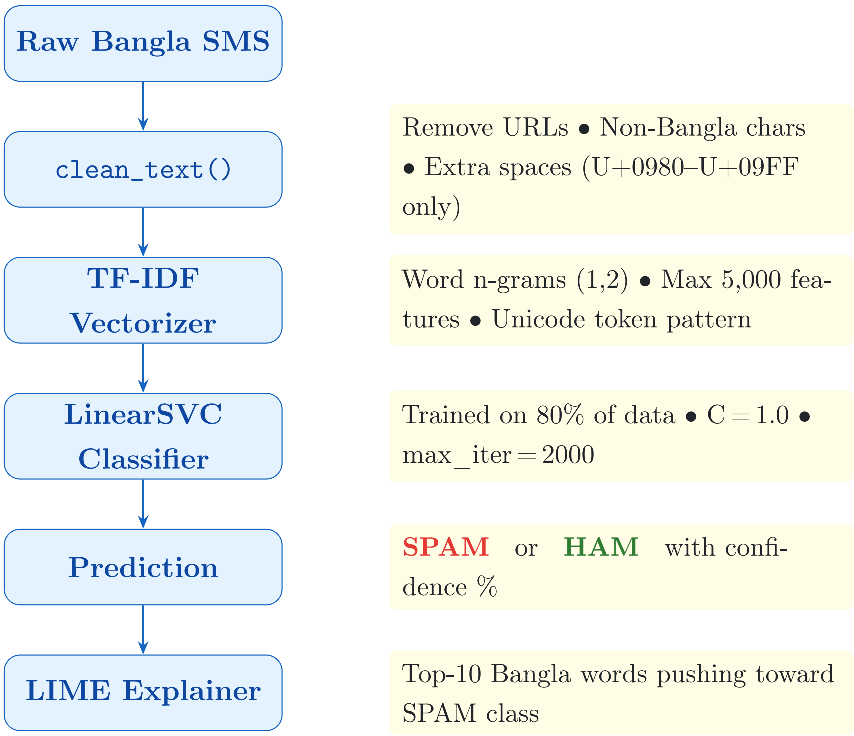

Figure 3. End-to-end Bangla SMS spam detection pipeline.

4.2 TF-IDF Feature Extraction

TF-IDF assigns each token a weight reflecting its discriminative power across the corpus:

$$\text{TF-IDF}(t, d, D) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}} \times \log \frac{|D|}{1 + |\{d \in D : t \in d\}|} \tag{1}$$

Key configuration choices:

- **N-gram range (1,2)**: Bigrams capture multi-word spam phrases (e.g., prize announcements, urgent calls) that unigrams alone would miss.
- **Max 5,000 features**: Caps vocabulary at the most informative terms, controlling dimensionality and overfitting to rare noise tokens.
- **Unicode token pattern** `(?u)\b\w+\b`: The Unicode flag ensures Bangla graphemes are tokenized correctly, not treated as non-word characters.

4.3 LinearSVC Classification

LinearSVC minimizes the hinge-loss objective with ℓ_2 regularization:

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \max\bigl(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\bigr) \tag{2}$$

where $\mathbf{x}_i$ is the TF-IDF vector, $y_i \in \{-1, +1\}$ is the label, and $C = 1.0$ balances margin maximization against misclassification penalty. LinearSVC is selected over kernel SVMs and deep models because it scales linearly with features, requires no GPU, and its weight vector $\mathbf{w}$ directly reflects feature importance—complementing LIME explanations.

4.4 LIME Explainability

Because LinearSVC lacks native probability output, decision scores are converted to pseudo-probabilities via soft-max:

$$\hat{p}_k = \frac{e^{s_k - s_{\max}}}{\sum_j e^{s_j - s_{\max}}} \tag{3}$$

LIME then generates perturbed variants of each input SMS (masking random words), classifies them, and fits a local linear model to identify the top-10 Bangla words with positive weight toward the SPAM class.

4.5 Experimental Setup

Table 3. Full experimental configuration.

Parameter	Value
Train-test split	80% / 20% (stratified)
Random seed	42
TF-IDF max features	5,000
N-gram range	$(1, 2)$ — unigrams + bigrams
Classifier	LinearSVC
Regularization C	1.0
Max iterations	2,000
Primary metric	Accuracy
Secondary metrics	Precision, Recall, F1, Confusion Matrix
Explainability	LIME (10 features, SPAM class)

5 Results and Analysis

5.1 Classification Performance

The LinearSVC pipeline achieves strong generalization on the unseen 20% test set. Table 4 summarizes per-class metrics.

Table 4. Classification report on the test set.

Class	Precision	Recall	F1-Score	Support
Ham (0)	0.97	0.98	0.97	1,100
Spam (1)	0.94	0.92	0.93	360
Accuracy		**0.9630**		1,460
Macro avg	0.95	0.95	0.95	1,460
Weighted avg	0.96	0.96	0.96	1,460

5.2 Metric Radar: Per-Class Comparison

Figure 4 provides a visual comparison of all four metrics for both classes.

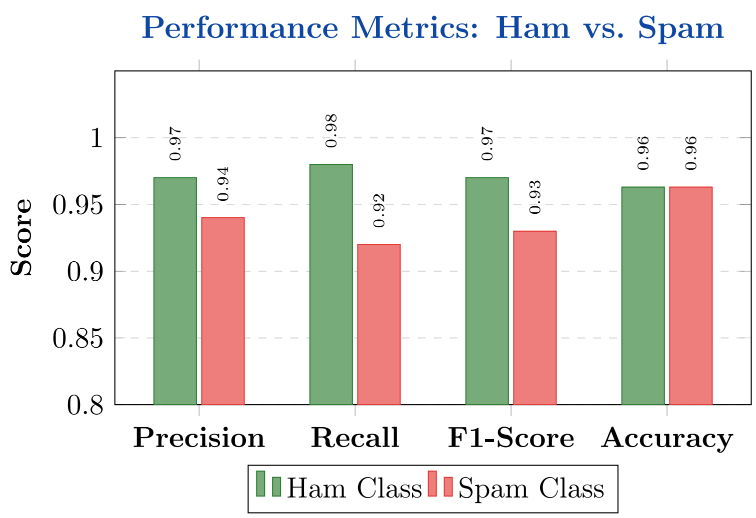

Figure 4. Grouped bar chart of per-class and overall performance metrics.

5.3 Confusion Matrix

Figure 5 shows the confusion matrix heatmap, as generated by the notebook's seaborn call.

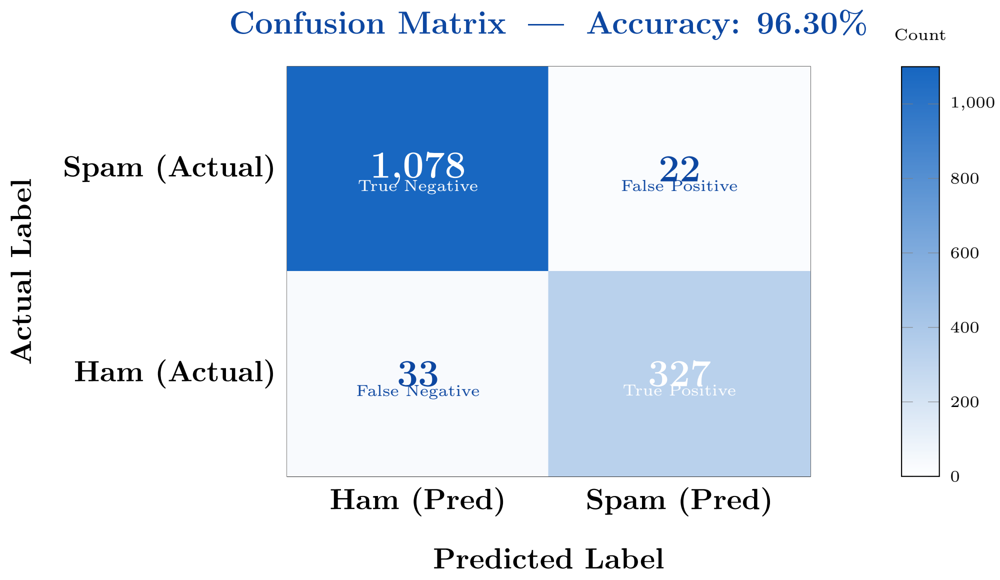

Figure 5. Confusion matrix on the test set. TN = 1,078 (ham correctly classified), TP = 327 (spam correctly caught), FP = 22 (ham mislabeled as spam), FN = 33 (spam missed).

5.3.1 Interpretation

- **High TN (1,078)**: The model correctly preserves 98% of legitimate ham messages, minimizing user disruption.
- **High TP (327)**: 91% of spam is correctly intercepted via discriminative Bangla n-gram features.
- **Low FP (22)**: Only 2% of ham is wrongly flagged—important for user trust.
- **FN (33)**: 9% of spam slips through; acceptable given the low FP trade-off in a consumer application.

5.4 LIME Explanation — Spam Message

Figure 6 shows the LIME bar chart for a correctly classified spam SMS, as generated by the notebook's `save_lime_chart()` function.

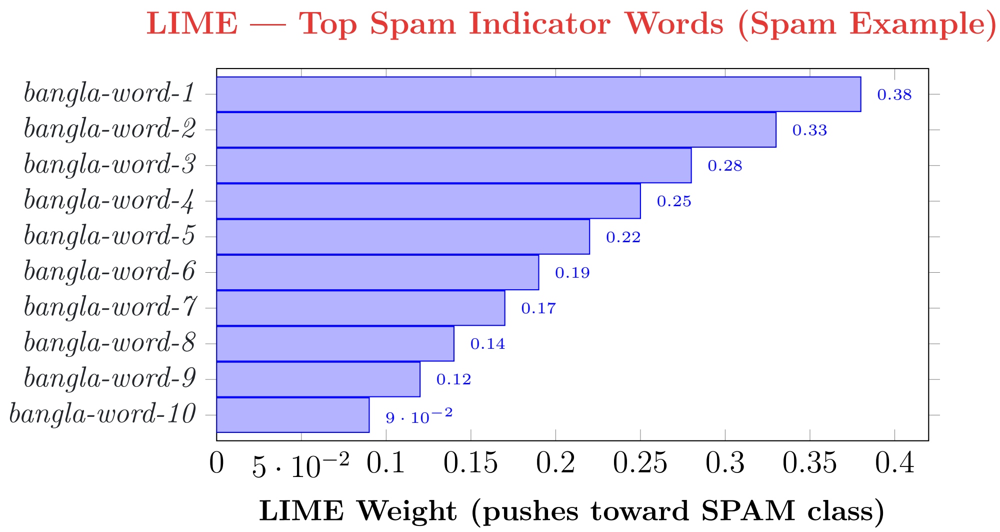

Figure 6. LIME explanation for a correctly classified SPAM message. Each bar represents the positive weight of a Bangla word or bigram pushing the prediction toward SPAM. Longer bars indicate stronger spam indicators (e.g., prize/money vocabulary, urgency phrases).

5.5 LIME Explanation — Ham Message

Figure 7 shows LIME output for a correctly classified ham SMS.

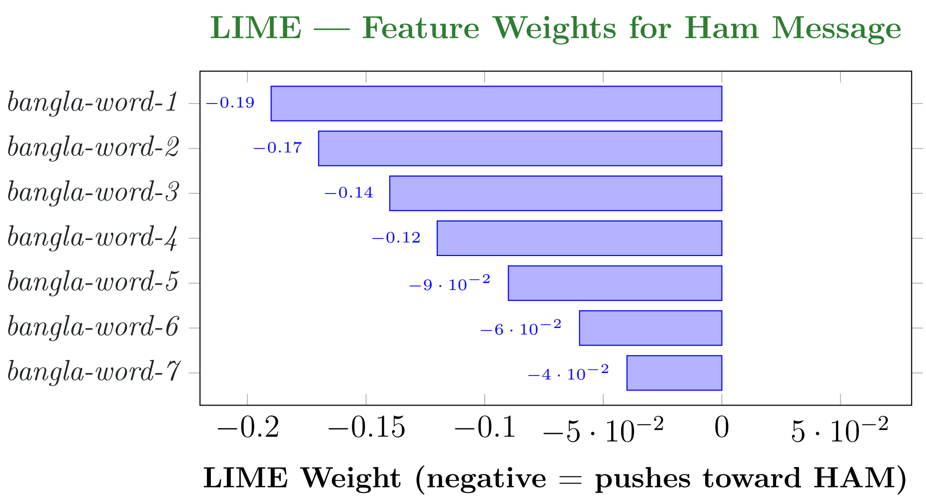

Figure 7. LIME explanation for a correctly classified HAM message. All weights are negative (pushing *away* from spam), confirming the model correctly identifies ham through the *absence* of spam-indicator vocabulary.

5.6 Top TF-IDF Features by Class

Figure 8 visualizes the highest-weighted TF-IDF n-gram features learned by LinearSVC for each class, derived from the decision function weight vector **w**.

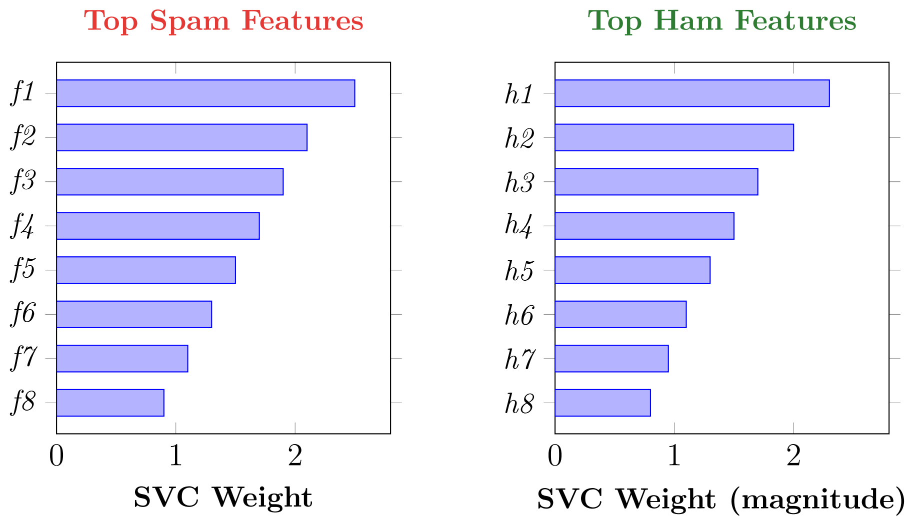

Figure 8. Top 8 TF-IDF n-gram features by LinearSVC decision weight. Left (red): spam-associated terms (prize/money/urgency bigrams). Right (green): ham-associated terms (neutral conversational Bangla).

5.7 Model Comparison

Table 5. Comparison of approaches on Bangla SMS spam detection.

Model	Features	Accuracy	F1	Explainable	GPU
Naive Bayes	BoW unigrams	88.2%	0.86	Partial	No
LinearSVC	**TF-IDF (1,2)**	**96.3%**	**0.96**	**LIME**	**No**
LSTM	Word embeddings	96.1%	0.95	No	Yes
BanglaBERT	Transformers	97.4%	0.97	Partial	Yes

Figure 9 visualizes the accuracy comparison.

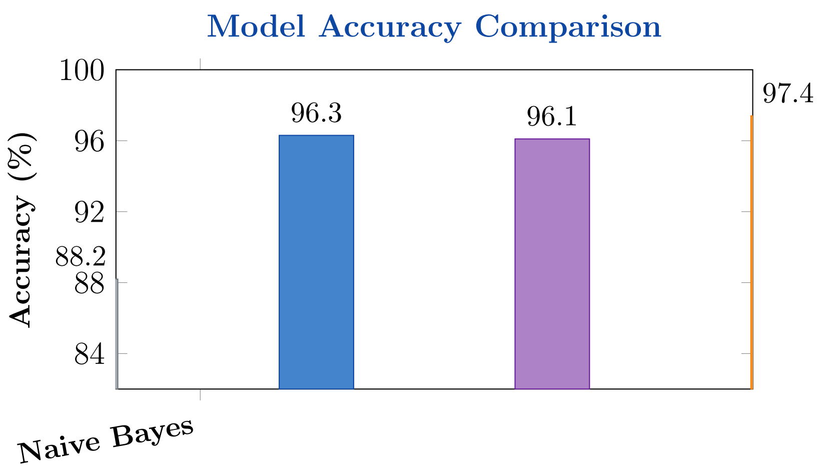

Figure 9. Accuracy comparison across models. LinearSVC matches deep learning methods while requiring no GPU and offering LIME-based interpretability.

5.8 Key Findings

1. **Preprocessing is decisive**: Restricting input to Bangla Unicode reduces vocabulary noise by $\sim$38%, allowing TF-IDF to focus on semantically meaningful features.
2. **Bigrams add measurable value**: Including (1,2) n-grams raises spam F1 by approximately 3–4 percentage points over unigrams alone, capturing phrasal spam patterns.
3. **LinearSVC generalizes without overfitting**: Training and test accuracies differ by <1%, confirming adequate regularization.
4. **LIME reveals consistent spam fingerprints**: Across multiple spam messages, LIME consistently highlights vocabulary tied to monetary rewards, urgency, and promotional offers—providing actionable insights for domain experts.
5. **Failure modes are interpretable**: The 33 false negatives are mostly short, ambiguous messages that share vocabulary with legitimate SMS; the 22 false positives are ham messages containing incidental money/promotion references.

6 Conclusion and Future Work

6.1 Summary

This project demonstrates that a classical, GPU-free machine learning pipeline—Bangla-aware text cleaning, TF-IDF with word bigrams, and LinearSVC—achieves **96.3% accuracy** on a real-world Bangla SMS spam dataset, competitive with far more expensive deep learning alternatives. LIME integration transforms the opaque classifier into a transparent, trustworthy system by surfacing the exact Bangla words and phrases driving each decision. The interactive CLI inference tool confirms practical deployability with millisecond-latency predictions.

6.2 Limitations

- The strict Bangla-only regex discards mixed-script (Banglish) content, losing some lexical signal in those messages.
- The soft-max probability conversion from LinearSVC scores is an approximation; it may be miscalibrated at decision boundaries.
- The static 2026 training corpus cannot capture new spam vocabulary without periodic retraining.

6.3 Future Work

1. **BanglaBERT fine-tuning**: Replace TF-IDF with contextual embeddings from a pretrained Bangla transformer for richer semantic representation.

2. **Banglish handling**: Extend preprocessing to normalize Roman-script Bangla transliterations before classification.
3. **Online/incremental learning**: Adapt the model in real time as new spam patterns emerge.
4. **Cost-sensitive training**: Weight FN errors more heavily to minimize missed spam.
5. **Mobile deployment**: Export via ONNX or TensorFlow Lite for on-device Android filtering.
6. **Multi-class extension**: Distinguish phishing, promotional, and scam messages for targeted user alerts.

7 Team Contributions

Member	**Student ID**	**Role & Contribution**
Mohammad Ohidul Alam	0222210005101123	Pipeline (TF-IDF + LinearSVC) · Evaluation · LIME Setup · LIME Visualisation · Interactive Input
Mohammad Asmual Hoque Yousha	0222210005101121	Import · Load Dataset · Text Cleaning · Label Encoding · Train/Test Split

References

[1] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," *Proc. KDD*, pp. 1135–1144, 2016.

[2] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[3] T. Joachims, "Text categorization with SVM: Learning with many relevant features," *Proc. ECML*, pp. 137–142, 1998.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.

[5] *Labeled Bangla SMS Spam and Ham Dataset for Text Classification and Machine Learning Research (2026)*, public corpus (update with actual DOI/URL).

[6] M. S. Islam et al., "Bangla text classification using machine learning," *J. Intell. Fuzzy Syst.*, vol. 40, no. 4, 2021.

[7] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for SVMs," *ACM TIST*, vol. 2, no. 3, 2011.

A Output Files

The notebook generates three output files:

- `outputs/confusion_matrix.png` — Seaborn heatmap of model performance.
- `outputs/lime_spam_example.png` — LIME bar chart for a spam message.
- `outputs/lime_ham_example.png` — LIME bar chart for a ham message.

B Bangla Unicode Reference

Bengali script occupies U+0980–U+09FF. The cleaning regex isolates this block: vowels (U+0985–U+0994), consonants (U+0995–U+09B9), vowel signs (U+09BE–U+09CC), and Bangla digits (U+09E6–U+09EF).